

# 49. Le garder dans l'écran

Un octet ne prévient pas quand il repasse par zéro.

## 1. Ce qu'il faut comprendre

Sans le `x > GAUCHE_MAX`, arriver à 0 puis décrémenter donne **255** : le personnage réapparaît à droite. C'est le bogue le plus fréquent de cette machine, et il ne fait jamais planter — il téléporte, silencieusement.

Tester **avant** de bouger, et non corriger après. `if (x < 8) x = 8;` marche aussi, mais laisse la variable fausse pendant une ligne — et cette ligne finit toujours par en attirer d'autres.

### Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **296 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

|                    |      |                                                                       |
|--------------------|------|-----------------------------------------------------------------------|
| démarrage          | 47 o | ← la console, à l'allumage — la mise en route de la console           |
| main()             | 77 o | ← démarrage — ta fonction                                             |
| VBlank             | 8 o  | ← la console, à chaque image — compte les images, 60 fois par seconde |
| AttendreImage      | 28 o | ← image() — attend l'image suivante                                   |
| AttendreVBlank     | 16 o | ← démarrage — attend que l'écran soit libre                           |
| RangerLutins       | 11 o | ← démarrage — range les 40 lutins hors de l'écran                     |
| InstallerTransfert | 15 o | ← démarrage — installe la copie des lutins en mémoire rapide          |
| CopierTuiles       | 31 o | ← démarrage — copie les lettres et les dessins en mémoire vidéo       |
| LireManette        | 37 o | ← bouton() — lit les huit boutons                                     |
| Tuiles             | 16 o | ← CopierTuiles — 0 lettre sur 43, et 1 tuile dessinée                 |
| DonneesTransfert   | 10 o | ← InstallerTransfert — la petite routine qui copie les lutins         |

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

|          |     |                                                   |
|----------|-----|---------------------------------------------------|
| sprite() | × 1 | 23 o sur place, dans main()                       |
| bouton() | × 2 | 10 o sur place, dans main() + LireManette (37 o)  |
| image()  | × 1 | 6 o sur place, dans main() + AttendreImage (28 o) |

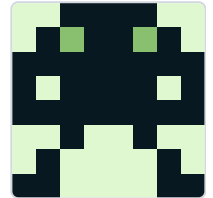
## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : déplace un personnage à gauche et à droite, sans jamais
// sortir de l'écran.
//
// On ne bouge QUE si on n'est pas déjà au bord :
//   if (bouton(GAUCHE) && x > GAUCHE_MAX) x--;
//   && = ET : la flèche est enfoncée ET x est plus grand que la limite.

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <bouton>    // lit un bouton de la manette
#include <sprite>    // place un lutin de 8 × 8 au pixel près
```

## 2. UN DESSIN — HEROS

```
Tuile HEROS = {
    "..###..", ".#-##-#.", "#####", "#.####.#",
    "#####", "..#..#..", ".#....#.", "##....##",
};
```



à l'écran

## 3. UNE CONSTANTE — GAUCHE\_MAX

```
const uint8_t GAUCHE_MAX = 8;    // Pas plus à gauche que le pixel 8.
```

## 4. UNE CONSTANTE — DROITE\_MAX

```
const uint8_t DROITE_MAX = 152;  // Pas plus à droite que 152 (160 - 8 : le lutin fait 8).
```

## 5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                      // Le jeu commence ici.
    uint8_t x = 76;                // La place du lutin, en pixels.

    while (true) {                // Répète pour toujours :
        if (bouton(GAUCHE) && x > GAUCHE_MAX) x--;  // à gauche, s'il reste de la place
        if (bouton(DROITE) && x < DROITE_MAX) x++;  // à droite, s'il reste de la place

        sprite(0, x, 68, HEROS);  // On repose le lutin.
        image();                  // Attend l'image suivante.
    }
}
```

# 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



**DROITE** — l'écran a changé



**GAUCHE** — l'écran a changé

## 5. Ce qu'on doit voir

Le personnage s'arrête aux bords au lieu de réapparaître en face.

## 6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ il s'arrête à gauche ( $x = 8$ )
- ✓ sans jamais déborder à 255
- ✓ il s'arrête à droite ( $x = 152$ )
- ✓ et le lutin suit

Ensuite : **50. Une vitesse qui monte** — chapitre 7 : Le mouvement. · gameboy3 — écrire une cartouche Game Boy en C++